

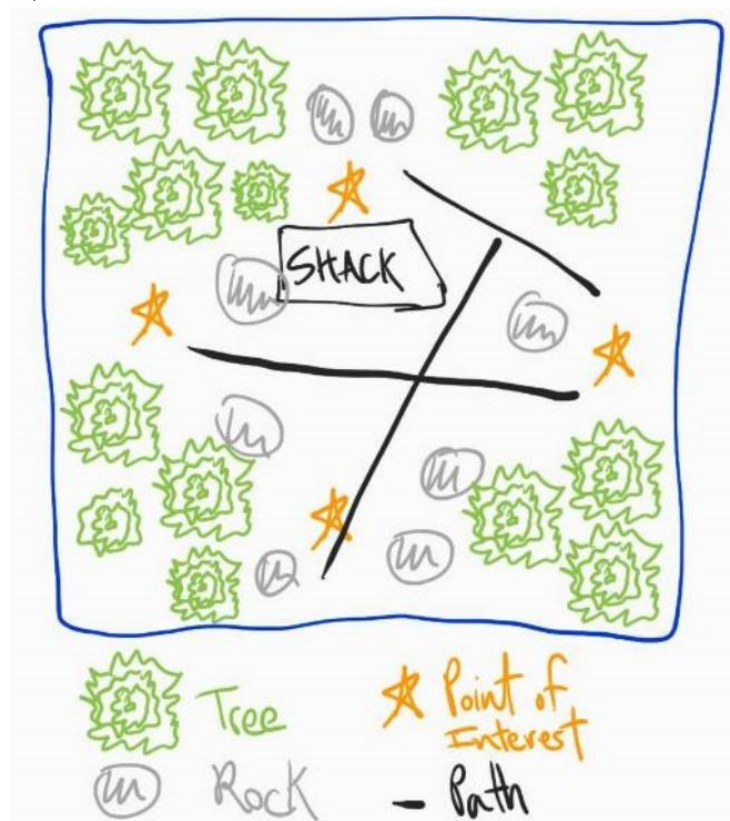
1. Objetivos

- Criar a cena inicial do Mundo do Game, “Despertar”
- Apresente o player à interface do usuário
- Mova o personagem
- Interaja com personagens não-jogadores
- Interaja com o meio ambiente

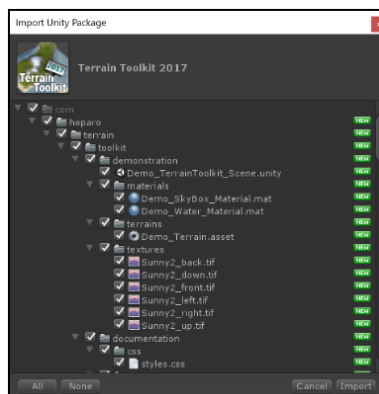
CENA INICIAL

2. Configurando o Mundo do Game

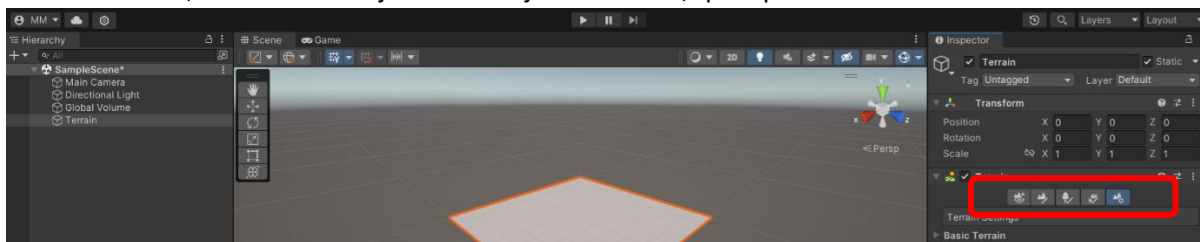
- Abrir um novo projeto no Unity, tipo 3D Standard (URP), renomeie de RPG.
- Vamos criar a Scene “Despertar”, de acordo com o rascunho:



- Importar um Asset para a cena do Mundo, em > Window > Package Management > Asset Store. Depois que o navegador abrir, busque por “Terrain Toolkit 2017”, e selecione “Add to my Assets”.
- No Unity a janela de Packages que se abrirá e clique no asset listado na ABA “My Assets” e faça o download do Terrain Toolkit 2017.
- Abrirá a tela do Import Unity Package, e clique em “Import”:



- f) Crie um terreno, em > Game Object > 3D Object > Terrain, que aparecerá mais ou menos assim:



- g) No box vermelho em destaque estão as ferramentas da componente terrain que podem ser utilizadas, para modificar a gosto o território, mas vamos usar o Toolkit instalado. Clique em > Component > Terrain > Terrain Toolkit e no Inspector surgirá um novo componente adicionado ao Game Object Terrain:



- h) As ferramentas ficam mais sofisticadas, e temos alguns tipos pré-definidos: Desert Mesa, Desert Sands, Sea Shore, Snowy Mounts, Pampa e Path of the Flesh.
i) Alguns detalhes deste toolkit:

Create

Pré-geradores definidos

- **Voronoi:** Isso cria uma altura arbitrária delineada de uma progressão de pináculos semelhantes a montanhas utilizando um contorno de Voronoi e aplica-o ao território
- **Fractal:** Isso produz um contorno de estatura arbitrário da nuvem ou fractal de cálculo, e aplica-o à paisagem
- **Perlin:** Isso gera uma altura irregular e a aplica ao território

Filtros:

- **Smooth:** Este é um canal que aplica suavização ao protesto de território mais de uma vez terminou várias ênfases.
- **Normalise:** Este é um canal que padroniza o protesto do território, definindo o ponto mais surpreendente na atual paisagem guia de estatura para o maior, e o menor ponto para a base. Cada outro ponto é adicionado entre o mais extremo e menos.

Erode (Erosão)

- **Erosão Térmica:** Isso expelle material de territórios com uma inclinação mais notável do que a inclinação da base, e armazena-a adicionalmente na inclinação. Isso tende a suavizar e endireitar as inclinações na paisagem.

- **Erosão Hidráulica:** Isso expõe o material de zonas com uma inclinação menor que a inclinação mais extrema e armazena-o adicionalmente na inclinação. Isso tende a acentuar as inclinações no território e suavizar e endireitar diferentes zonas.
- **Erosão das marés:** Aplica a suavização no nível do oceano escolhido, exceto em territórios muito inclinados. Isso modela a atividade erosiva das ondas em torno de uma linha costeira e produzindo bordas costeiras.

Texture

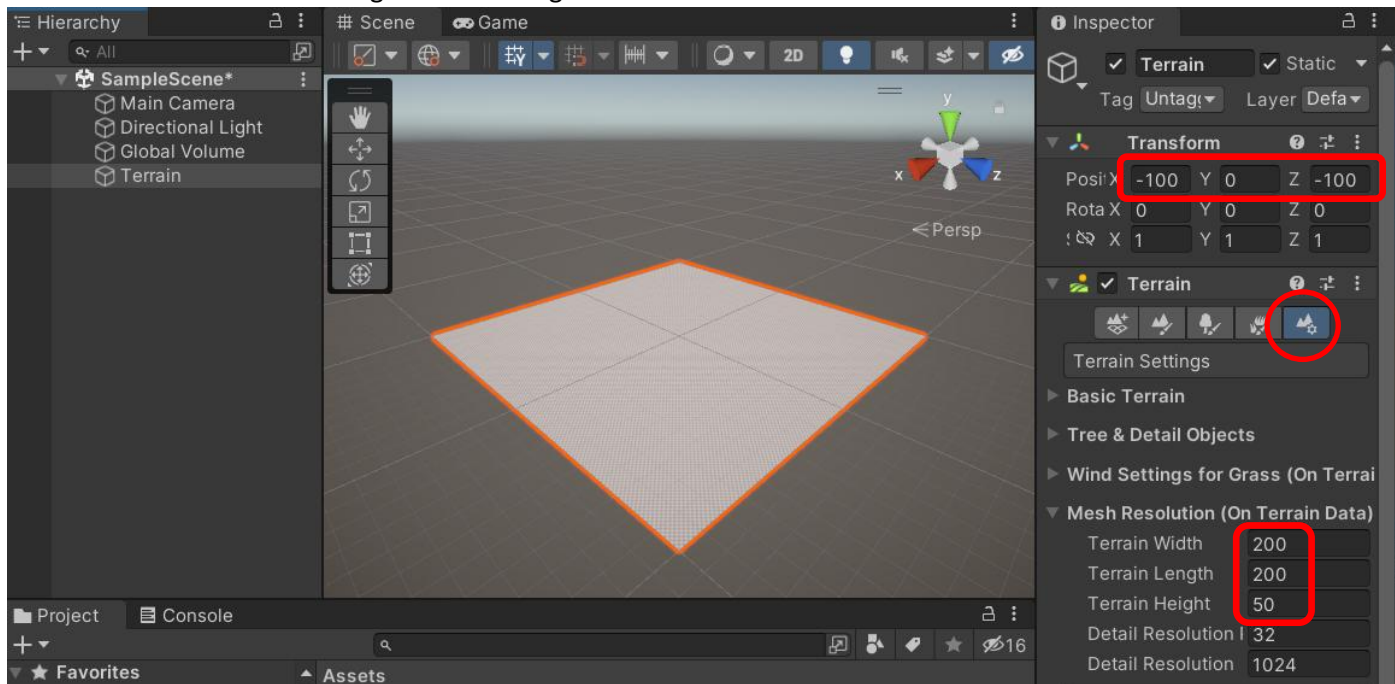
O passo final é aplicar a texturização. Isso dará ao terreno uma aparência e sensação mais realistas em tempo de execução. O Toolkit fornece texturização de terreno processual, que texturiza automaticamente o objeto de terreno.

3. Cena “DESPERTAR”

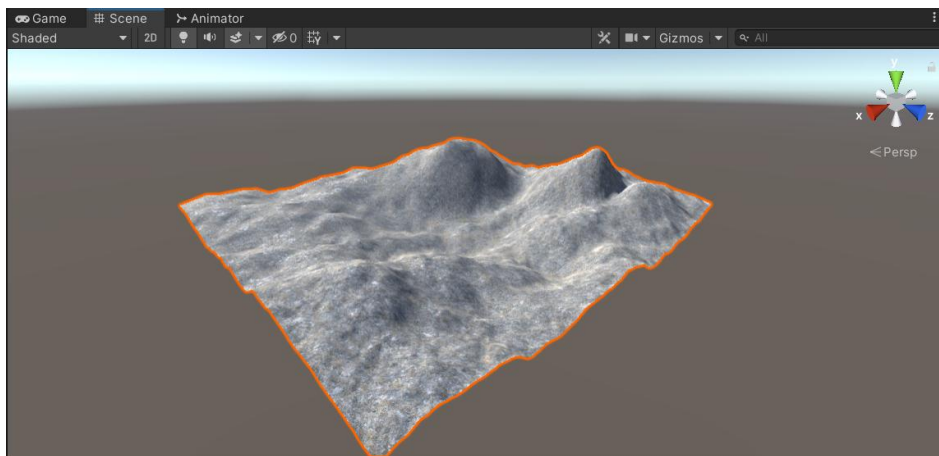
- Clicando duas vezes no Game Object Terrain na Hierarchy, ele ficará centrado na Scene.
- Por default ele é em geral bem grande, e assim, ajuste o tamanho no Inspector em Terrain > Transform:
 - > Position x = -100, Y = 0, Z = -100
- Selecione Terrain Settings, e em Mesh Resolution (On Terrain Data),:



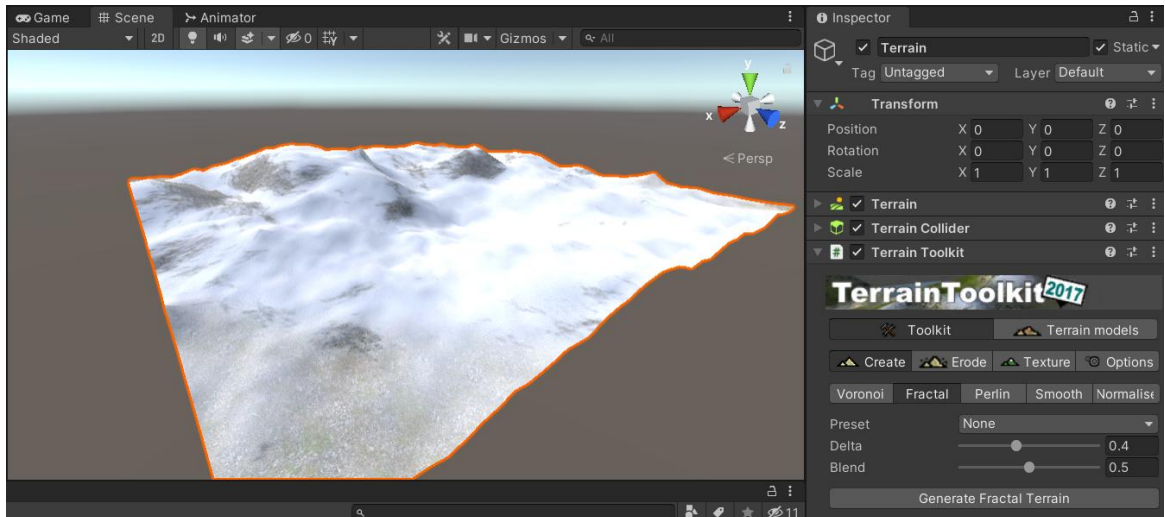
-> Width = Length = 500 e Height 30



- Selecione Terrain na Hierarchy, e clique em > Component > Terrain > Terrain Toolkit. No Inspector Terrain Models > SNOWY MOUNTS e deixe-o gerar o território. Por exemplo uma geração pode ser assim:

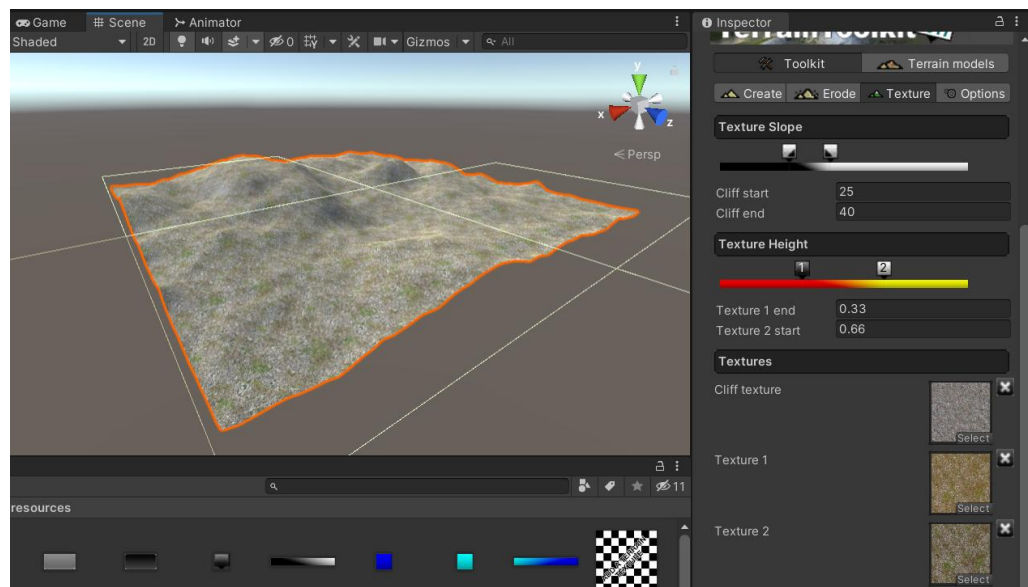


- e. Usando o filtro fractal em > Toolkit > Fractal (Delta 0.4 e Blend 0.5) (a imagem pode não ser a mesma...):

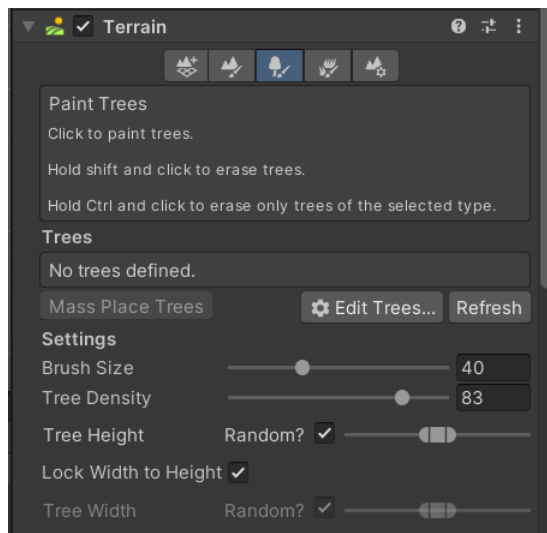


Filtros diferentes produzem outras suavizações.

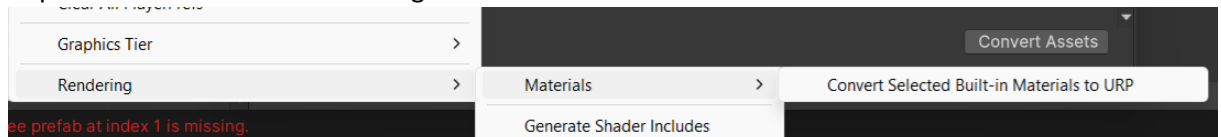
- f. Adicionando texturas, de acordo com o nível do terreno, podemos ter até 4, e no caso deste exemplo, com 3:



- g. Usando a ferramenta de Terrain, para popular trees:

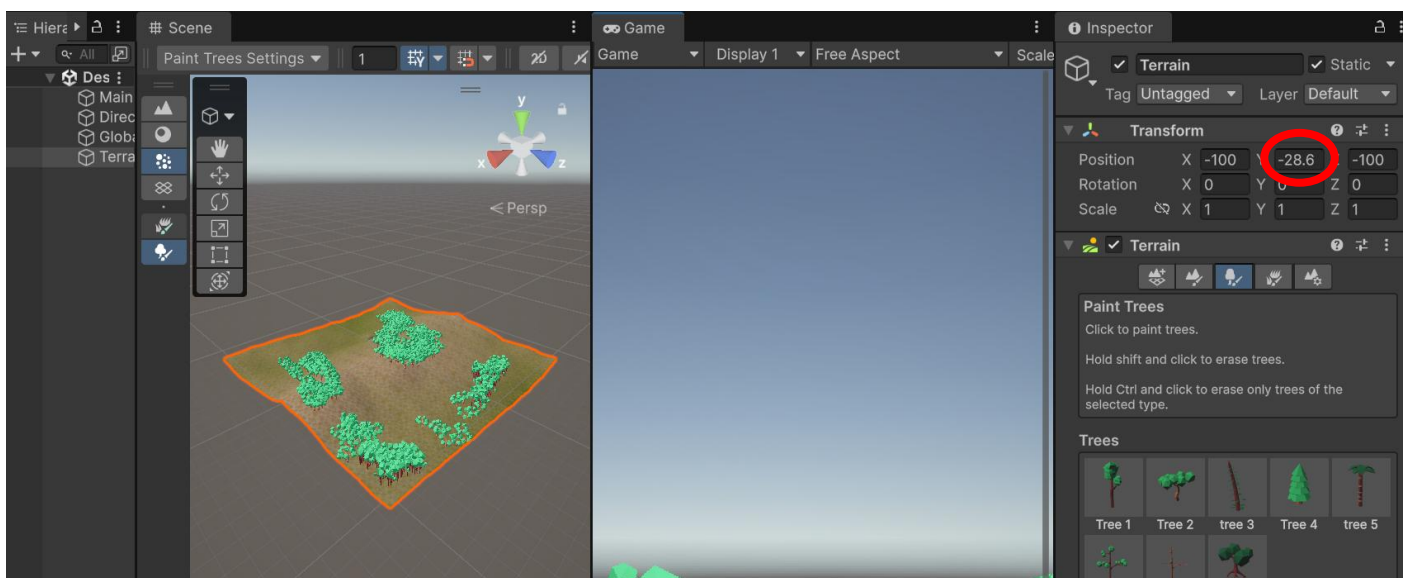


- h. Selecione uma tree e pinte com o pincel, a vontade. Caso não encontre alguma árvore, busque na asset store > Window > Asset Store > Low Poly Stylized Nature.
- i. Se precisar (ficarem rosa os prefabs), acerte materials em > Window > Rendering > Render Pipeline Converter > Material Upgrade. Depois clique em Initialize Converters e depois clique em Convert Assets.
- j. Se ainda permanecerem “rosa” os prefabs, precisa buscar em cada pasta > Materials do package importado e em > Edit > Rendering > Materials > Convert Selected Built-in Materials to URP



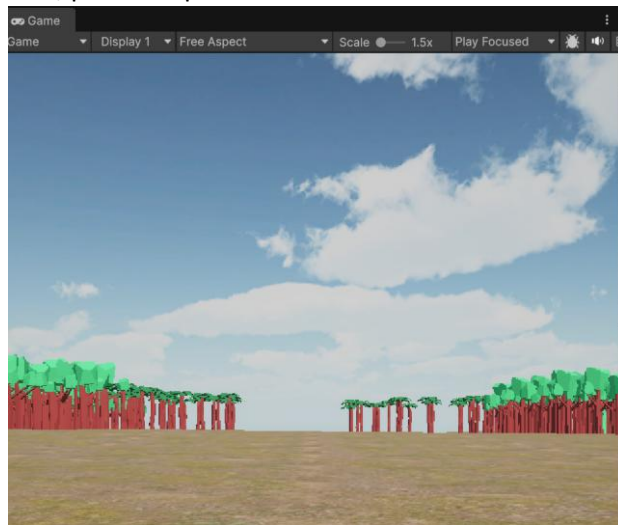
É o preço pelo asset “gratuito”...!

- k. Na opção > Paint Trees (terceira do Inspector de Terrain) adicione as árvores > Add Tree e busque os prefabs no pacote importado no > Assets.
- l. Escolha um tipo de pincel (Paint Options) e opacidade, pinte as árvores, como no rascunho.
- m. No meu caso ficou assim, sendo que “abaixei” o Terrain em Pos XYZ = (-100,-28.6,-100),
- n. Main Camera perto do centro do terrain Pos XYZ = (25, 2, 32):



- 4. **Skybox.** Equivale a colocar nosso mundo numa caixa, e forrá-la por dentro com uma aparência do céu.

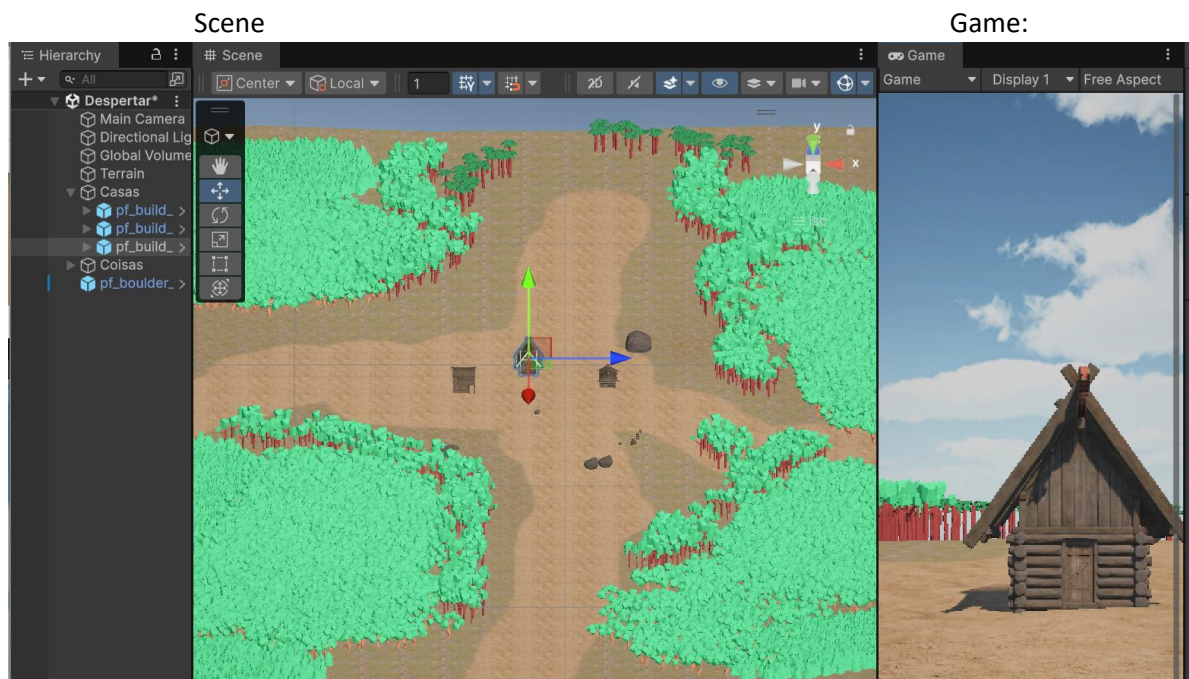
- a. Buscando na Asset Store um sky box free (p.ex. Fantasy Skyboxes FREE), adicionando em My Assets e importando.
- b. Selecione em Hierarchy Main Camera e adicione a Componente Skybox em > Add Component > Skybox. Em Custom Skybox, selecione uma delas, e depois configure em Transform XYZ para acertar a posição do sol e nuvens, na ABA Game, por exemplo:



MONTANDO UMA CABANA ISOLADA NA CLAREIRA

5. Pedras e Cabana:

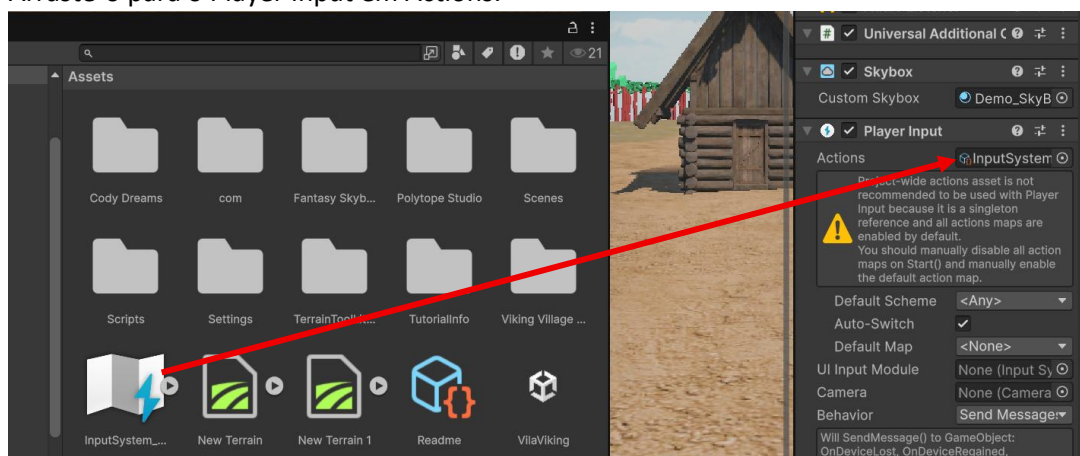
- a. Importe o asset Low Poly Rock Pack (acerte os Materials).
- b. Importe o asset Viking Village URP (idem materials), sendo que neste segundo package, deve clicar em Switch Project para não perder seu trabalho e abrir nova seção do Unity, pegar só os prefabs que desejar e voltar ao projeto inicial. Veja que pode esta Scene importada já pode ser “a final” adaptada ao nosso rascunho do RPG.
 - i. Selecione VikingVillage em \Assets e exporte um pacote, tomando o cuidado de tirar a seleção das Scenes, pois não precisaremos delas em nosso projeto. Renomeie de VilaViking.
 - ii. Importe este pacote VilaViking para o projeto inicial.
- c. Escolha algumas pedras e algumas cabanas posicionando no mundo como no rascunho. No meu caso ficou algo como:



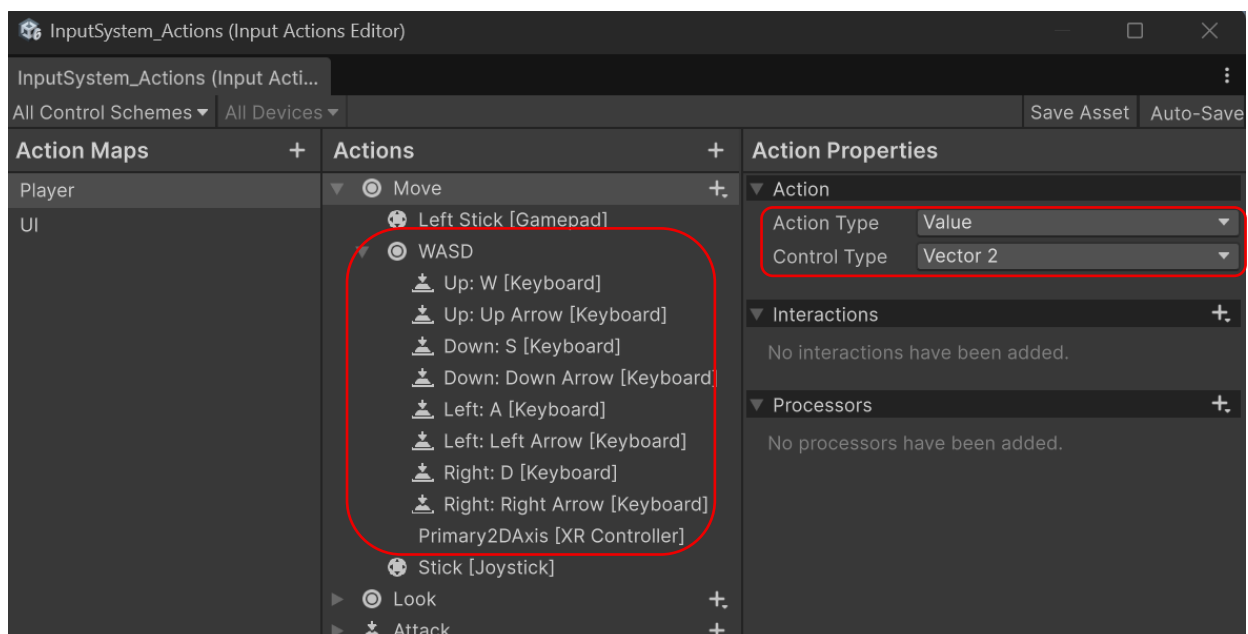
- d. Para navegar no cenário, escreva um script no estilo FPS, chamado de “MovePlayer.cs” e adicione-o na Main Camera, salvando-o numa pasta \Scripts:

```
1  g UnityEngine;
2  g UnityEngine.InputSystem;
3
4  0 references | Unity Script
5  ic class MovePlayer : MonoBehaviour
6
7  1 reference | Unity Serialized Field
8  public float velocidade = 5f;
9  3 references
10 private Vector2 moveInput;
11
12 0 references
13 public void OnMove(InputValue valor)
14 {
15     /// <summary
16     /// Este método é chamado automaticamente pelo novo sistema de entrada do Unity
17     ///
18     moveInput = valor.Get<Vector2>();
19 }
20
21 0 references | Unity Message
22 void Update()
23 {
24     Vector3 movimento = new Vector3(moveInput.x, 0f, moveInput.y);
25     transform.Translate(movimento * velocidade * Time.deltaTime, Space.World);
26 }
```

- e. Perceba que vamos usar o sistema novo de entrada do Unity, o New Input System. A interface é a função OnMove(), que no caso vai nos fornecer as coordenadas x e y do player, pelas teclas de movimentação WASD e as direcionais.
- f. Precisa adicionar à Main Camera o componente Player Input, via Inspector > Add Component > Player Input
- g. Abra na ABA Project, localize o componente InputSystem_Actions:
- i. Arraste-o para o Player Input em Actions:



- ii. Clique duas vezes no InputSystem_Actions e
1. Verifique ao clicar na sub-aba “Player” e depois em “Move”, surgindo os campos de “Action Properties”, se aparecem:

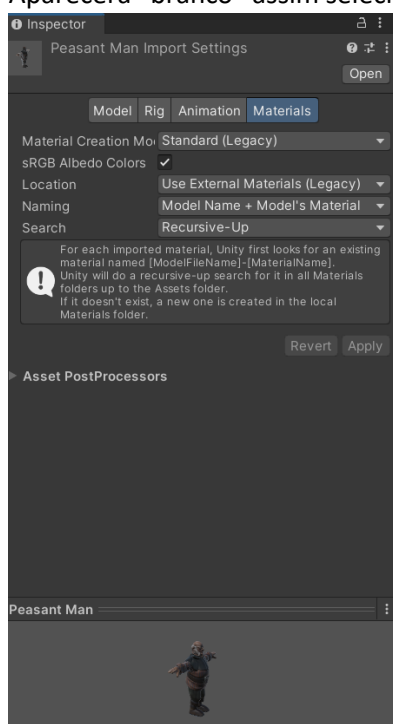


- iii. No script do MovePlayer o método “OnMove” irá recolher as interações com as teclas WASD e direcionais $\uparrow \leftarrow \downarrow \rightarrow$
- h. Rode e teste, verificando se a câmera (player) se desloca no cenário com as teclas WASD e com as $\uparrow \leftarrow \downarrow \rightarrow$.
- i. Salve a Scene.

INSERINDO NPC

6. NPC's de interação.

- a. Em Mixamo.com, busque o Character “Peasant Man”, faça download dele, em T-Pose, no formato FBX for Unity, depois importe este asset numa pasta \Prefabs, renomeando-o de “Aldeao”.
- i. Aparecerá “branco” assim selecione-o e no Inspector, > Materials, selecione:

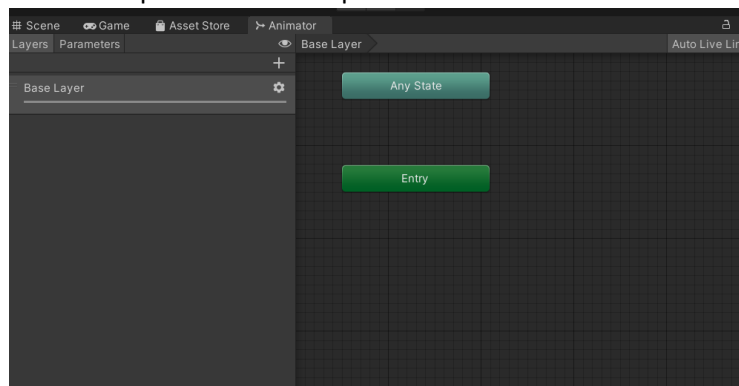


- ii. Que os material serão carregados adequadamente.
- iii. Ajuste a escala dele para ficar no tamanho adequado para a casa, por exemplo Scale XYZ = (1.2, 1.2, 1.2)
- iv. Adicione a este prefab um Capsule Colider:
 - 1. ajuste Center Y para 1, para ficar centrado em todo caractere.
 - 2. Ajuste Height para 1.5

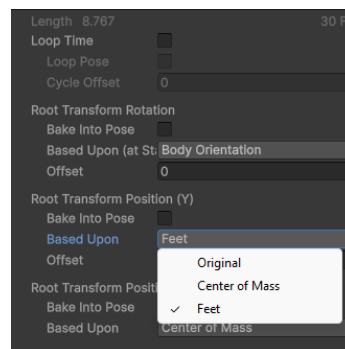
- b. Em Mixamo também, busque as animações “Neutral Idle”, “Sitting Angry”, “Sitting Talking” (sugiro baixar esta com Skin), “Sit to Stand”, “Talking” e “Stand to Sit”, e faça download de todas, importando-as no seu projeto, numa pasta \Animations.
- c. Arraste o prefab homem1 para a Hierarchy, perto da cabana. Ajuste para ele ficar de frente, como:



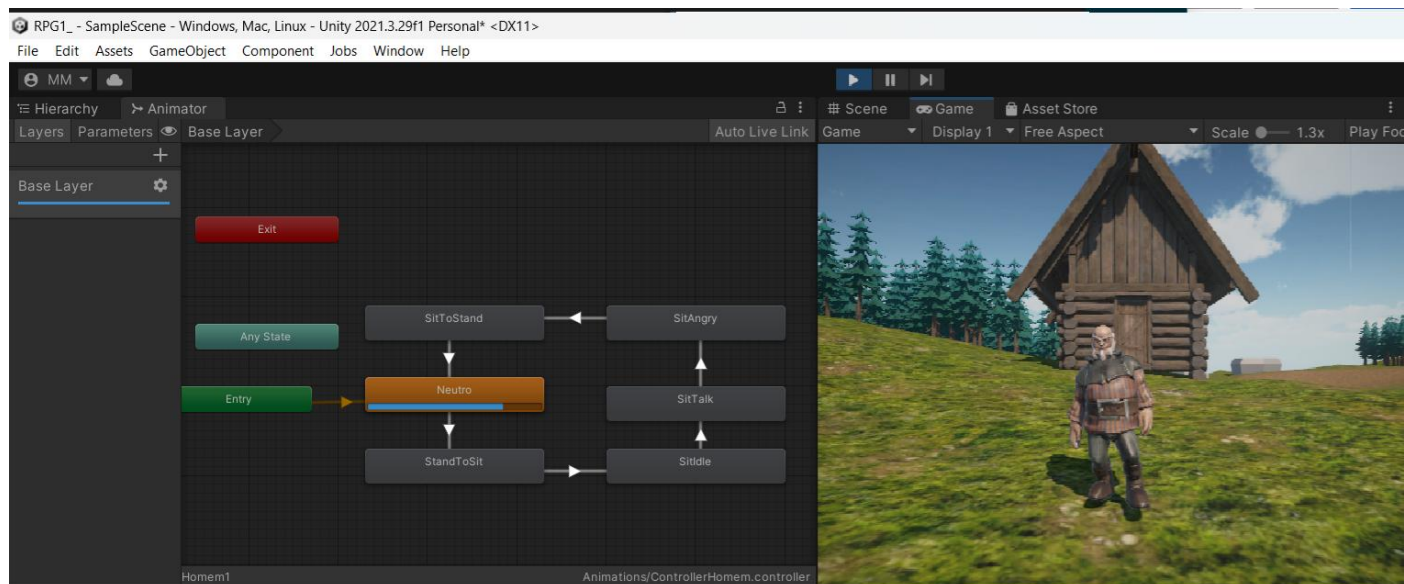
- d. Crie na aba de Project um Animator Controller, renomeando de ControllerHomem. Arraste este controlador para a o Aldeao. Clique nele e deve aparecer assim:



- e. Arraste um a um cada animação que fez download, para este controller, renomeando cada uma (aparece mixamo.com) para não perder a ordem, depois interligue todas, clicando com o botão direito do mouse e arrastando as transições, numa ordem.
- f. Execute e veja o que ocorre:
 - i. Se o caractere “voar” em alguma animação, é preciso acertar isso na importação da mesma, no inspector:
 1. Selecione a animação com problema em \Projects e em Root Transform Position (Y) selecione > Based Upon “Feet”:



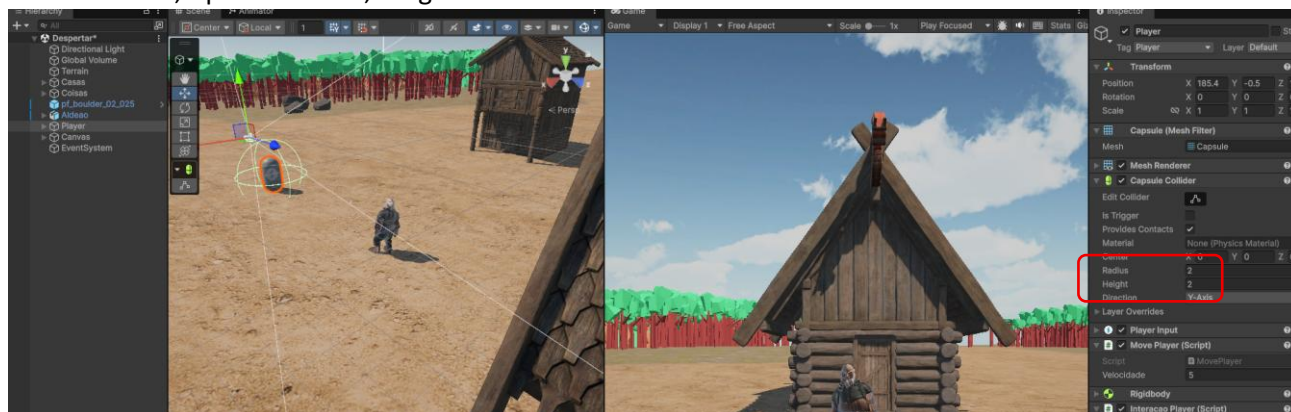
- ii. Rode novamente, e corrija o posicionamento de cada animação que tiver algum problema.



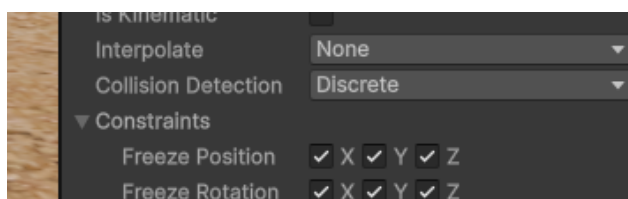
- g. Deixe agora como “Default” a animação “Sitting Talking” e retire todas as outras transições. Vamos cuidar do comportamento do Aldeão mais para frente.
- h. Perceba que falta uma “pedra” para nosso homem sentar-se. Busque uma no projeto e posicione-a para que ele não sente no “ar”.
- i. Salve a Scene.

INTERAGINDO COM O NPC ALDEÃO

7. Vamos fazer o Player interagir com o NPC Aldeão. A ideia é fazer uma detecção de proximidade, o Aldeão inicia sentado e “percebe” o Player se aproximando e se levanta; o Player interage e o Aldeão diz algo em resposta.
 - a. Para ter um collider, insira na Hierarchy um 3D Object Capsule, que será nosso Player, que até agora era apenas a Main Camera.
 - b. Renomeie-o de Player na Hierarchy
 - c. Coloque uma tag nele de “Player”, via Inspector
 - d. Faça Main Camera ser Filho deste novo Player
 - i. Retire o script MovePlayer de Main Camera e passe para este novo Player agora
 - ii. Faça o mesmo com a componente Player Input
 - e. Clique no Player e na Aba Scene veja as configurações deste Capsule Collider, melhor ajustar para ser maior, tipo Radius = 2, Height = 2



- f. Adicione um componente Rigidbody ao Player e acerte todas as Constrains, para ele não “cair” ou “girar”:



- g. Rode e teste se o Player se movimenta OK.

- h. Crie o script “AldeaoDialogo.cs” e anexe-o ao Aldeao na Hierarchy. Iremos trabalhar nele em seguida, mas por hora isso já evita erro de compilação.
- i. Criaremos dois scripts adicionais para o Player, começando por “InteracaoPlayer.cs”, que conecta via trigger o Player com o script do Aldeão e libera-o quando sair da região do collider. Anexe-o ao Player:

```

1  using UnityEngine;
2  2 references
3  public class InteracaoPlayer : MonoBehaviour
4  {
5      5 references
6      public AldeaoDialogo atualNPC;
7      0 references
8      private void OnTriggerEnter(Collider outro)
9      {
10         AldeaoDialogo npc = outro.GetComponent<AldeaoDialogo>();
11         // O trigger de colisão dispara este método que configura o atualNPC como o Aldeão
12         if (npc != null)
13         {
14             atualNPC = npc;
15         }
16     }
17
18     0 references
19     private void OnTriggerExit(Collider outro)
20     {
21         AldeaoDialogo npc = outro.GetComponent<AldeaoDialogo>();
22         // Quando o Player afasta-se do Aldeão, libera outras interações com o player
23         if (npc != null && npc == atualNPC)
24         {
25             atualNPC = null;
26         }
27     }
28 }

```

8. O próximo script do Player é com o novo sistema de Input. A **Ideia central no novo Input System**, você não fica perguntando “qual tecla está pressionada” a cada frame:
 - a. 🖱 Em vez disso, você **escuta eventos**. Ou seja:
 - i. O input **acontece**
 - ii. O sistema **te avisa**
 - b. É necessário o uso de **OnEnable** e **OnDisable**, que controlam **quando seu script começa e para de ouvir inputs**:
 - i. ✅ **OnEnable()** é chamado quando o objeto/script é ativado
 1. 🖱 Ativa o sistema de input
 2. conecta os eventos
 - ii. ❌ **OnDisable()** é chamado quando o objeto/script é desativado
 1. 🖱 desativa input
 2. remove os eventos (IMPORTANTE!)
 - c. O New Input System tem Actions, e cada Action tem fases, sendo uma delas performed:

Fase	Quando acontece
started	começou (ex: tecla apertada)
performed	ação válida aconteceu
canceled	ação terminou (tecla solta)

- d. Assim, no script PlayerInputHandler.cs:

```

1 using UnityEngine;
2 using UnityEngine.InputSystem;
  0 references
3 public class PlayerInputHandler : MonoBehaviour
4 {
  4 references
5     public InputAction interagir;
  3 references
6     private InteracaoPlayer interacao;
  0 references
7     private void Awake()
8     {
9         interacao = GetComponent<InteracaoPlayer>();
10    }
11
  0 references
12    private void OnEnable()
13    {
14        interagir.Enable();
15        interagir.performed += OnInteract;
16    }
17
  0 references
18    private void OnDisable()
19    {
20        interagir.performed -= OnInteract;
21        interagir.Disable();
22    }
23
  2 references
24    private void OnInteract(InputAction.CallbackContext context)
25    {
26        if (interacao.atualNPC != null)
27        {
28            interacao.atualNPC.Interagir();
29        }
30        else
31        {
32            Debug.Log("Nenhum NPC por perto");
33        }
34    }
35 }

```

- e. Neste script a InputAction interagir precisa ser configurada, via Inspector, pois temos que avisar qual entrada será usada, no caso a tecla “E”. Antes contudo, deve aparecer uma mensagem de erro, pois o método “Interagir()” ainda não existe no script AldeaoDialogo.
- i. Resolva isso escrevendo um public void Interagir() vazio dentro de “AldeaoDialogo.cs”, para possibilitar a compilação:

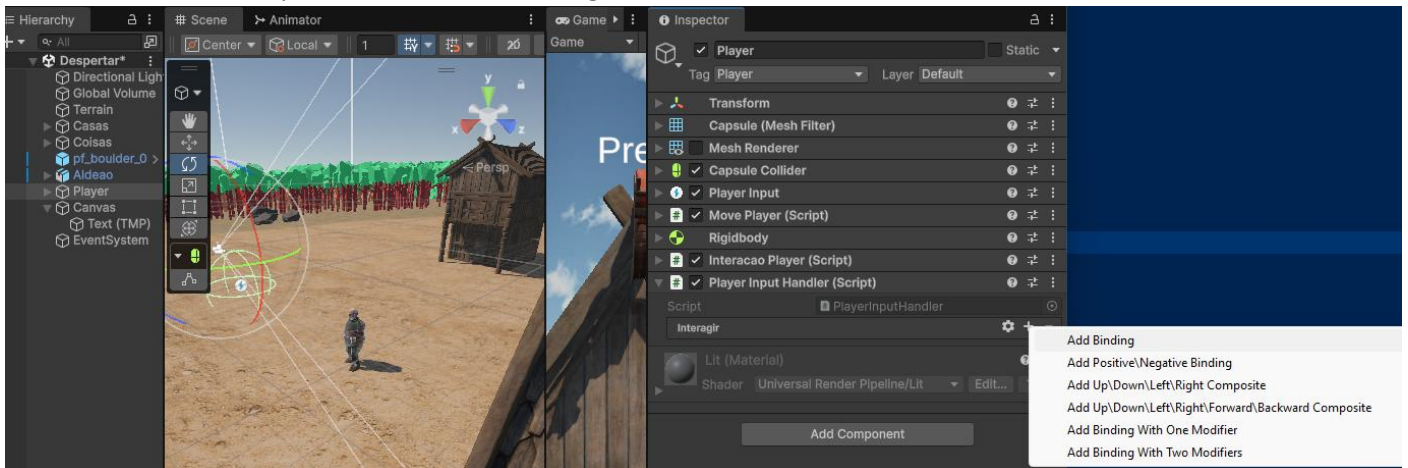
```

1 using UnityEngine;
2
  5 references
3 public class AldeaoDialogo : MonoBehaviour
4 {
  1 reference
5     public void Interagir()
6     {
7
8     }
9 }

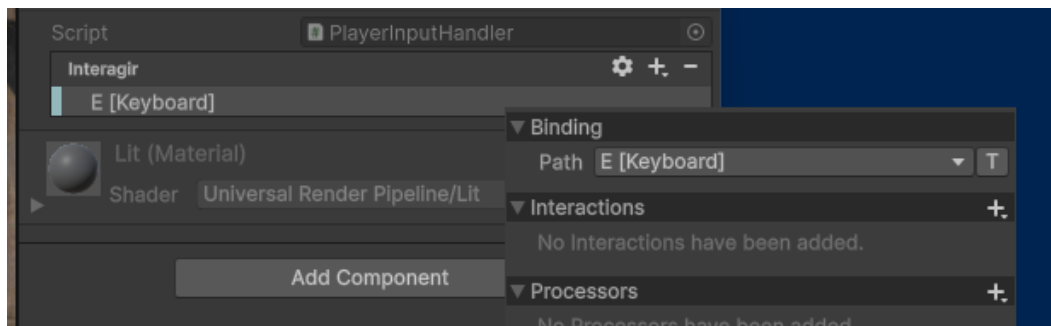
```

- f. Vamos configurar a entrada para chamar Interagir:

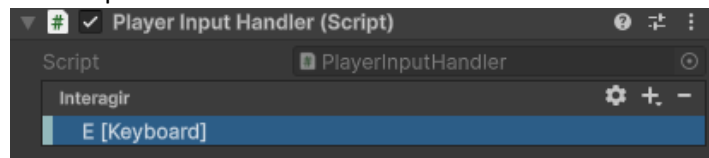
- i. Selecione o Player, e no Inspector de Player Input Handler deve aparecer a função Interagir, clique em “+” e > Add Binding:



- ii. Clique em <No Binding> e deve aparecer um pop up e em “Path” você pode ir escolhendo entrada até chegar na tecla “E”, mas há uma maneira simples, clicando no botão “Listen” (o sistema estará aguardando a entrada...) depois tecele a tecla E e confirme Keyboard E, deve estar assim ao final deste menu:



- iii. E fica assim no Inspector:



- iv. Poderia ser Gamepad, Joystick ou outra interface...
- v. Para testar, insira no script “AldeaoDialogo.cs” um print de debug:

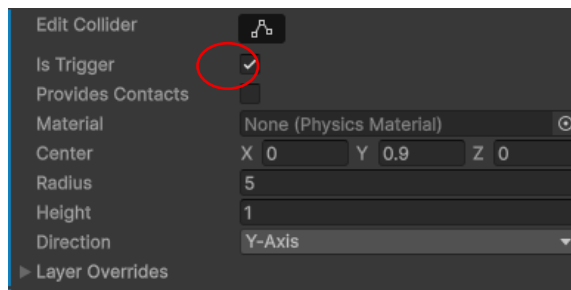
```

1 using UnityEngine;
2
3 5 references
4 public class AldeaoDialogo : MonoBehaviour
5 {
6     1 reference
7     public void Interagir()
8     {
9         Debug.Log("Tecleou E!");
10    }
11 }

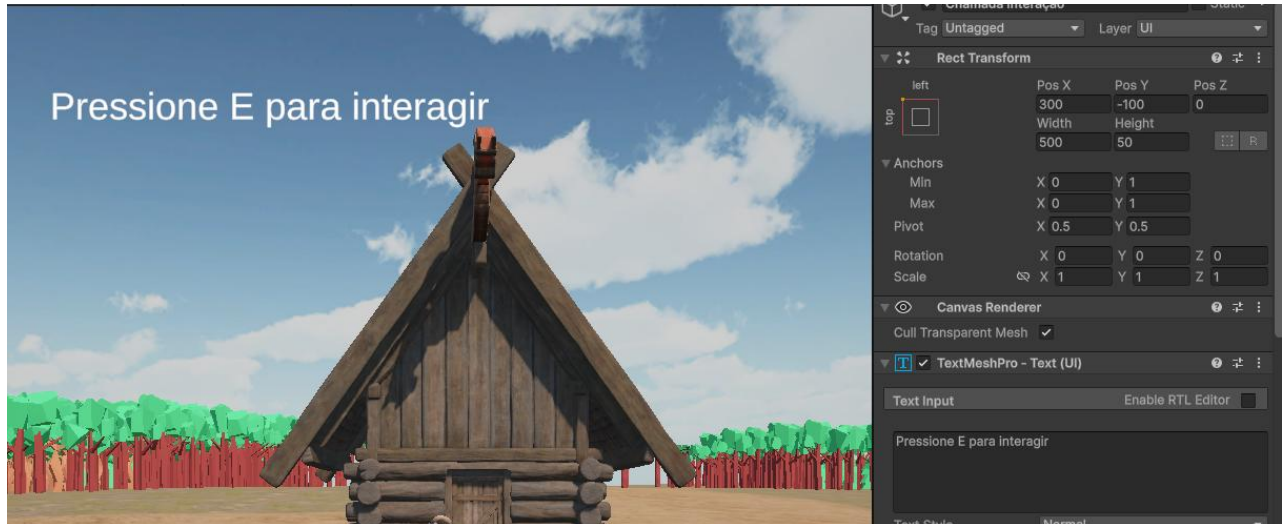
```

9. Vamos configurar agora o Aldeão para ter disparar o trigger adequado.

- a. Selecione-o na Hierarchy. Adicione um Capsule Collider generoso, para o Player não ter de se aproximar muito:
 - i. Center XYZ = (0, 0.9, 0)
 - ii. Radius = 5
 - iii. Height = 1
- b. Selecione isTrigger, para permitir a ativação dos eventos



- c. Na Hierarchy adicione um Canvas Text TMP em Botão direito Mouse > UI (Canvas) > Text Mesh Pro
 - i. Se não estiver instalado, o Unity abre uma janela para instalar este Package
 - ii. Renomeie na Hierarchy este TMP de “Chamada Interação”
- d. Selecionando este TMP no Canvas na Hierarchy, no box que aparecer no Inspector digite o texto “Pressione E para interagir”.
- e. Posicione a mensagem num ponto legal, por exemplo no alto:



- f. Crie o script “InteracaoAldeao.cs” e anexe-o ao Aldeao na Hierarchy:
 - i. Vamos criar um prompt de interação (nosso Canvas TMP) para avisar ao Player como interagir (tecla E), ativado e desativado na presença do Player
 - ii. Quando trigger do collider detectar, o NPC vai girar na direção do Player
 - iii. Haverá um print de que o NPC detectou o Player
 1. Acionamento de uma variável booleana “isPlayer” para mudar o estado de animação do Aldeão
 - iv. Outro print que o Player saiu da área de detecção
 1. Reset da variável booleana “isPlayer”

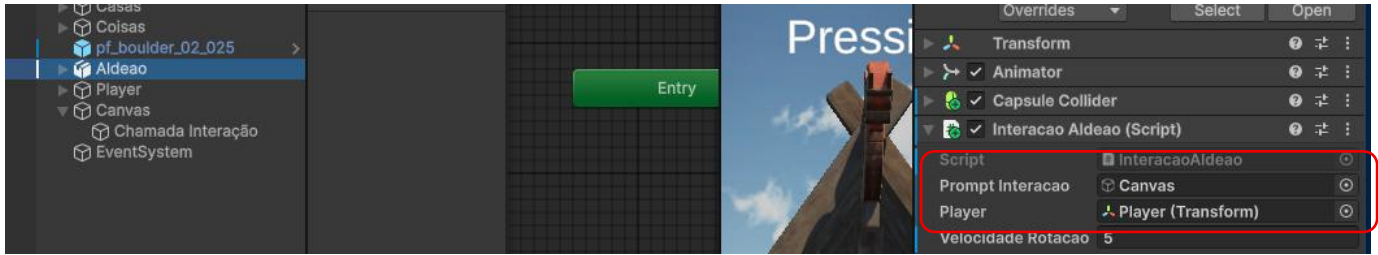
```

1  using UnityEngine;
   0 references
2  public class InteracaoAldeao : MonoBehaviour
3  {
   6 references
4  public GameObject promptInteracao; // ex.: texto "Pressione E para interagir"
   4 references
5  public Transform player;
   3 references
6  Animator anim;
   1 reference
7  public float velocidadeRotacao = 5f;
   3 references
8  private bool playerPerto = false;
9
   0 references
10 private void Start()
11 {
12     anim = GetComponent<Animator>();
13     if (promptInteracao != null)
14         promptInteracao.SetActive(false);
15 }
16
   0 references
17 private void OnTriggerEnter(Collider outro)
18 {
19     if (outro.CompareTag("Player"))
20     {
21         playerPerto = true;
22         player = outro.transform;
23         if (promptInteracao != null)
24             promptInteracao.SetActive(true);
25         Debug.Log("Player entrou na área do NPC.");
26         anim.SetBool("isPlayer", true);
27     }
28 }
29
30 private void OnTriggerExit(Collider outro)
31 {
32     if (outro.CompareTag("Player"))
33     {
34         playerPerto = false;
35         player = null;
36         if (promptInteracao != null)
37             promptInteracao.SetActive(false);
38         Debug.Log("Player saiu da área do NPC.");
39         anim.SetBool("isPlayer", false);
40     }
41 }
42
   0 references
43 private void Update()
44 {
45     if (playerPerto && player != null)
46     {
47         // Faz o NPC olhar para o player
48         Vector3 direcao = player.position - transform.position;
49         direcao.y = 0f;
50         if (direcao != Vector3.zero)
51         {
52             Quaternion targetRotation = Quaternion.LookRotation(direcao);
53             transform.rotation = Quaternion.Slerp(
54                 transform.rotation,
55                 targetRotation,
56                 velocidadeRotacao * Time.deltaTime
57             );
58         }
59     }
60 }
61 }

```

- g. Selecione Aldeao na Hierarchy e no Animator Controller dele crie em "Parameters" uma variável Bool "isPlayer"

- h. Posicione o Player fora da área de interação com o Aldeão.
- i. No Inspector coloque uma tag no Aldeão de “Aldeao”
 - i. Associe na interface Prompt Interacao o “Canvas” da Hierarchy
 - ii. Associe na interface Player o Player da Hierarchy



- j. Rode e teste. As mensagens no console e o Canvas de Interação devem funcionar quando o Player se movimenta perto do Aldeão e interage com ele teclando E.

10. Abra o script “AldeaoDialogo.cs”:

- a. Apague o método Interagir();
- b. Reescreva tudo:

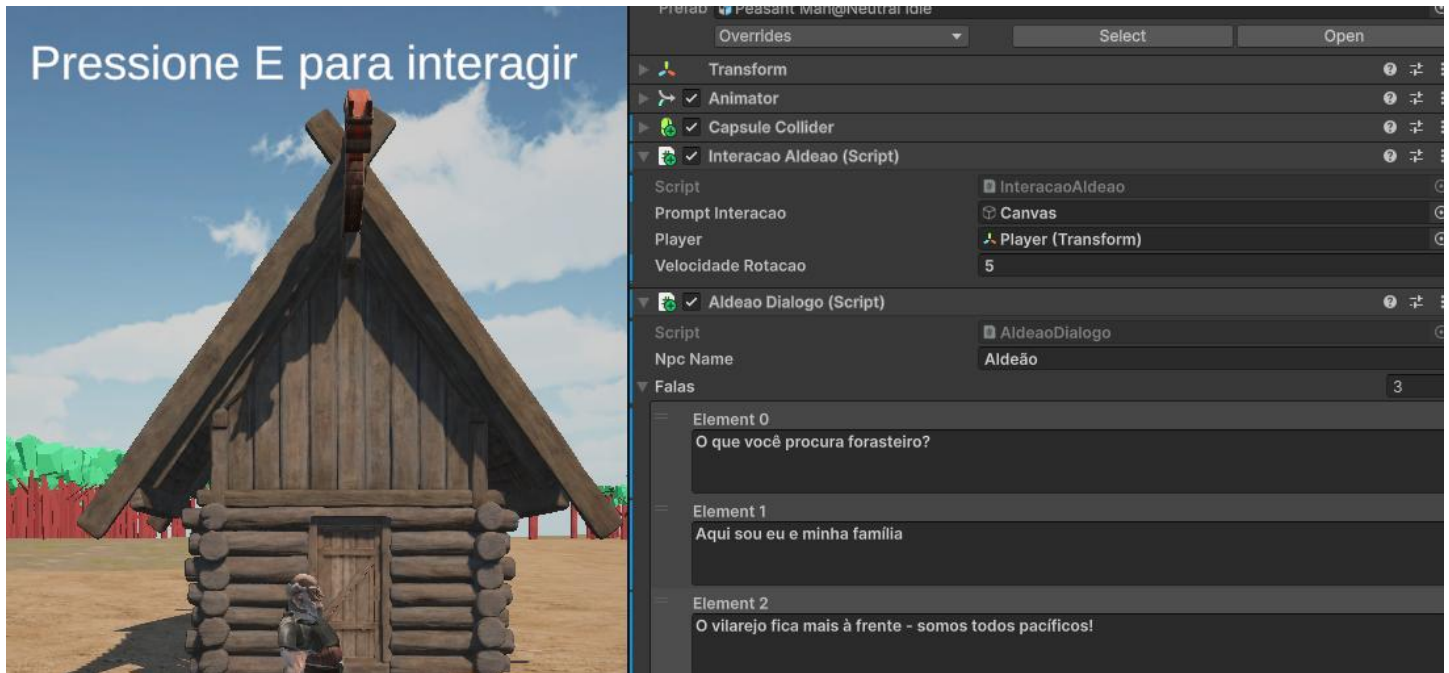
```

1  using UnityEngine;
   5 references
2  public class AldeaoDialogo : MonoBehaviour
3  {
4      1 reference
       public string npcName = "Aldeão";
       4 references
5      Animator anim;
       3 references
6      [TextArea] public string[] falas;
7
8      4 references
       private int index = 0;
9
10     0 references
       void Start()
11     {
12         anim = GetComponent<Animator>();
13     }
14
15     1 reference
       public void Interagir()
16     {
17         if (falas.Length == 0)
18         {
19             anim.SetBool("isTalking", false);
20             return;
21         }
22         anim.SetBool("isTalking", true);
23         Debug.Log(npcName + ": " + falas[index]);
24
25         index++;
26
27         if (index >= falas.Length)
28         {
29             index = 0;
30             anim.SetBool("isTalking", false);
31         }
32     }
33 }

```

- c. Depois de compilar, ao selecionar o Aldeao na Hierarchy deve aparecer a List da variável “Falas”:
 - i. Clique em “+” e adicione a primeira em Element 0: “O que você procura forasteiro?”

- ii. Novamente “+” e em Element 1: “Aqui sou eu e minha família”
- iii. “+” Element 2: “O vilarejo fica mais à frente: somos todos pacíficos!”
- iv. Etc...



- d. Adicione no Animator Controller do Aldeão o parâmetro bool “isTalking”
- e. Rode e teste, verifique se o Aldeão interagem com o Player quando este chega perto e, a cada tecla E pressionada ele “fala” através do console.
- f. Salve a Scene.

ANIMAÇÕES DO ALDEÃO

11. Vamos configurar o Animator Controller do Aldeão, para que ele:
 - a. inicie sentado murmurando algo,
 - b. se levante quando o Player se aproximar,
 - c. pergunta e gesticula ao Player,
 - d. fica em Idle esperando o Player interagir,
 - e. sente-se quando o Player se afasta.
12. Configure:
 - a. Transição de “Sitting Talking” -> “Sit To Stand”, com a Conditions, “isPlayer” = true, SEM Has Exit Time
 - b. Transição “Sit To Stand” -> “Talking”, com “isTalking” = true, SEM Has Exit Time
 - c. Transição “Talking” -> “Neutral Idle”, COM Has Exit Time
 - d. Transição “Neutral Idle” -> “Talking”, com “isTalking” = true, SEM Has Exit Time
 - e. Transição “Neutral Idle” -> “Stand To Sit”, COM Has Exit Time
 - f. Transição “Sit To Stand” -> “Stand To Sit”, com “isPlayer” = false, SEM Has Exit Time
 - g. Transição “Stand To Sit” -> “Sitting Talking”, COM Has Exit Time
13. Rode e teste. Salve a Scene.
14. Salve a Scene “Despertar”.

15. Lição de Casa:

- a. Inclua a animação “Sitting Angry” se o Player se afasta sem o Aldeão ter falado todas as suas frases, e depois voltar ao estado “Sitting Talking”
- b. Procure na Asset Store algo que possa ser a distração do Aldeão que fala sozinho: um animal, um boneco, uma estátua, etc.

-X-X-X-